Future Work Explained Pipeline: complete YouTube upload script

Repository: https://github.com/MAmmaadTehseen/faceless-business (public). Files below are the complete, working upload path. Run: python -m pipeline.run_all --publish --privacy private. The owner authorises once with scripts/youtube_auth.py; the refresh token is kept as a secret. Uploads go to the owner's own channel only.

== pipeline/publish/youtube.py ==

YouTube Data API client: upload, thumbnails, stats, analytics, comments, replies

```
"""YouTube Data API v3: upload, thumbnail, update title/thumbnail, analytics, comments."""
from __future__ import annotations

import os
from datetime import datetime, timedelta, timezone
from pathlib import Path

from google.oauth2.credentials import Credentials
from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload

from ..common import log

SCOPES = ["https://www.googleapis.com/auth/youtube", "https://www.googleapis.com/auth/youtube.force-ssl",
          "https://www.googleapis.com/auth/yt-analytics.readonly"]


def _creds() -> Credentials:
    return Credentials(None, refresh_token=os.environ["YT_REFRESH_TOKEN"],
token_uri="https://oauth2.googleapis.com/token",
                       client_id=os.environ["YT_CLIENT_ID"], client_secret=os.environ["YT_CLIENT_SECRET"],
scopes=SCOPES)


def yt():
    return build("youtube", "v3", credentials=_creds(), cache_discovery=False)


def yt_analytics():
    return build("youtubeAnalytics", "v2", credentials=_creds(), cache_discovery=False)


def configured() -> bool:
    return all(os.getenv(k) for k in ("YT_CLIENT_ID", "YT_CLIENT_SECRET", "YT_REFRESH_TOKEN"))


def upload(video: Path, title: str, description: str, tags: list[str], privacy: str = "private",
           publish_at: datetime | None = None, thumbnail: Path | None = None, is_short: bool = False) -> dict:
    body = {
        "snippet": {"title": title[:100], "description": description[:4900], "tags": tags[:15], "categoryId":
"28",
                    "defaultLanguage": "en", "defaultAudioLanguage": "en"},
        "status": {"privacyStatus": privacy, "selfDeclaredMadeForKids": False, "containsSyntheticMedia": True},
    }
    if publish_at and privacy == "private":
        body["status"]["publishAt"] = publish_at.astimezone(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
    media = MediaFileUpload(str(video), chunksize=8 * 1024 * 1024, resumable=True, mimetype="video/mp4")
    req = yt().videos().insert(part="snippet,status", body=body, media_body=media)
    resp = None
    while resp is None:
        status, resp = req.next_chunk()
    vid = resp["id"]
    if thumbnail and thumbnail.exists():
        try:
            yt().thumbnails().set(videoId=vid, media_body=MediaFileUpload(str(thumbnail),
mimetype="image/jpeg")).execute()
        except Exception as e:  # thumbnails need a verified phone on the channel
            log.warning("thumbnail set failed (verify the channel's phone number in Studio): %s", e)
    log.info("youtube: uploaded %s as %s (%s)", vid, privacy, "short" if is_short else "long")
    return {"id": vid, "url": f"https://youtu.be/{vid}" if not is_short else
f"https://youtube.com/shorts/{vid}"}


def update_title(video_id: str, title: str) -> None:
    v = yt().videos().list(part="snippet", id=video_id).execute()["items"][0]
    v["snippet"]["title"] = title[:100]
    yt().videos().update(part="snippet", body={"id": video_id, "snippet": v["snippet"]}).execute()


def set_thumbnail(video_id: str, path: Path) -> None:
    yt().thumbnails().set(videoId=video_id, media_body=MediaFileUpload(str(path),
mimetype="image/jpeg")).execute()
```

```
  67
  68
  69  def stats(video_ids: list[str]) -> dict[str, dict]:
  70      out = {}
  71      for i in range(0, len(video_ids), 50):
  72          chunk = video_ids[i:i + 50]
  73          res = yt().videos().list(part="statistics,status", id=",".join(chunk)).execute()
  74          for it in res.get("items", []):
  75              s = it["statistics"]
  76              out[it["id"]] = {"views": int(s.get("viewCount", 0)), "likes": int(s.get("likeCount", 0)),
  77                               "comments": int(s.get("commentCount", 0)), "privacy":
it["status"]["privacyStatus"]}
  78      return out
  79
  80
  81  def analytics(video_ids: list[str], days: int = 28) -> dict[str, dict]:
  82      """CTR, impressions, avg view duration per video from the Analytics API."""
  83      if not video_ids:
  84          return {}
  85      end = datetime.now(timezone.utc).date()
  86      start = end - timedelta(days=days)
  87      try:
  88          res = yt_analytics().reports().query(
  89              ids="channel==MINE", startDate=start.isoformat(), endDate=end.isoformat(),
  90              metrics="views,averageViewDuration,impressions,impressionClickThroughRate",
  91              dimensions="video", filters="video==" + ",".join(video_ids[:200]),
  92          ).execute()
  93      except Exception as e:
  94          log.warning("analytics query failed: %s", e)
  95          return {}
  96      cols = [c["name"] for c in res.get("columnHeaders", [])]
  97      out = {}
  98      for row in res.get("rows", []):
  99          d = dict(zip(cols, row))
 100          out[d["video"]] = {"avg_view_duration_sec": d.get("averageViewDuration"), "impressions":
d.get("impressions"),
 101                             "ctr": d.get("impressionClickThroughRate")}
 102      return out
 103
 104
 105  def comments(video_id: str, max_results: int = 50) -> list[dict]:
 106      try:
 107          res = yt().commentThreads().list(part="snippet", videoId=video_id, maxResults=max_results, order="time",
textFormat="plainText").execute()
 108      except Exception as e:
 109          log.warning("comments fetch failed for %s: %s", video_id, e)
 110          return []
 111      out = []
 112      for it in res.get("items", []):
 113          top = it["snippet"]["topLevelComment"]
 114          s = top["snippet"]
 115          out.append({"platform_comment_id": top["id"], "author": s.get("authorDisplayName"), "text":
s.get("textDisplay", ""),
 116                      "posted_at": s.get("publishedAt"), "is_own": s.get("authorChannelId", {}).get("value") ==
_my_channel_id()})
 117      return out
 118
 119
 120  _channel_cache: dict = {}
 121
 122
 123  def _my_channel_id() -> str:
 124      if "id" not in _channel_cache:
 125          res = yt().channels().list(part="id", mine=True).execute()
 126          _channel_cache["id"] = res["items"][0]["id"]
 127      return _channel_cache["id"]
 128
 129
 130  def reply(parent_comment_id: str, text: str) -> str:
 131      res = yt().comments().insert(part="snippet", body={"snippet": {"parentId": parent_comment_id,
"textOriginal": text[:9000]}}).execute()
 132      return res["id"]
```

== pipeline/publish/__init__.py ==

publish_all: uploads the long video and its Shorts, records ids

```
  1  """Publish a rendered content item (long + shorts) to every configured platform."""
  2  from __future__ import annotations
  3
  4  from datetime import datetime, timedelta, timezone
  5  from pathlib import Path
  6
  7  from ..common import log, now_iso
  8  from . import meta, youtube
```

```
 9
10
11  def _next_slot(db, kind: str) -> datetime:
12      """Stagger: long videos 12h apart starting 1h from now; shorts 2h after their parent."""
13      now = datetime.now(timezone.utc)
14      rows = db.select("publications", {"platform": "youtube"}, order="publish_at", desc=True, limit=20)
15      future = [datetime.fromisoformat(r["publish_at"].replace("Z", "+00:00")) for r in rows if
r.get("publish_at")]
16      future = [t for t in future if t > now]
17      base = max(future) + timedelta(hours=12) if future else now + timedelta(hours=1)
18      return base
19
20
21  def publish_all(db, content_id: str, privacy: str = "private") -> list[dict]:
22      long = db.one("content", {"id": content_id})
23      shorts = db.select("content", {"parent_id": content_id, "kind": "short"})
24      results = []
25      long_url = None
26
27      if youtube.configured():
28          slot = _next_slot(db, "long")
29          try:
30              r = youtube.upload(Path(long["file_path"]), long["title"], long["description"] or "",
long.get("tags") or [],
31                                 privacy=privacy, publish_at=slot if privacy == "private" else None,
thumbnail=Path(long["thumbnail_path"]) if long.get("thumbnail_path") else None)
32              long_url = r["url"]
33              db.upsert("publications", {"content_id": content_id, "platform": "youtube", "platform_id": r["id"],
"url": r["url"], "privacy": privacy,
34                                         "publish_at": slot.isoformat(), "status": "live", "published_at":
now_iso()}, on="content_id,platform")
35              results.append({"platform": "youtube", "content_id": content_id, **r})
36          except Exception as e:
37              log.error("youtube long upload failed: %s", e)
38              db.upsert("publications", {"content_id": content_id, "platform": "youtube", "status": "failed",
"error": str(e)[:500]}, on="content_id,platform")
39      else:
40          log.warning("youtube not configured; skipping")
41
42      for i, s in enumerate(shorts):
43          desc = (s["description"] or "").replace("{LONG_URL}", long_url or "")
44          db.update("content", {"id": s["id"]}, {"description": desc})
45          if youtube.configured():
46              try:
47                  r = youtube.upload(Path(s["file_path"]), s["title"], desc, long.get("tags") or [],
privacy=privacy,
48                                     publish_at=(slot + timedelta(hours=2 * (i + 1))) if privacy == "private" else
None, is_short=True)
49                  db.upsert("publications", {"content_id": s["id"], "platform": "youtube", "platform_id": r["id"],
"url": r["url"], "privacy": privacy,
50                                             "publish_at": (slot + timedelta(hours=2 * (i + 1))).isoformat(),
"status": "live", "published_at": now_iso()}, on="content_id,platform")
51                  results.append({"platform": "youtube", "content_id": s["id"], **r})
52              except Exception as e:
53                  log.error("youtube short upload failed: %s", e)
54                  db.upsert("publications", {"content_id": s["id"], "platform": "youtube", "status": "failed",
"error": str(e)[:500]}, on="content_id,platform")
55          if meta.configured() and privacy == "public":
56              caption = f"{s['title']}\n\n{desc}".strip()
57              try:
58                  url = meta.public_url_for(Path(s["file_path"]),
tag=f"media-{datetime.now(timezone.utc):%Y%m%d}")
59                  r = meta.publish_instagram_reel(url, caption)
60                  db.upsert("publications", {"content_id": s["id"], "platform": "instagram", "platform_id":
r["id"], "url": r.get("url"), "privacy": "public",
61                                             "status": "live", "published_at": now_iso()},
on="content_id,platform")
62                  results.append({"platform": "instagram", "content_id": s["id"], **r})
63              except Exception as e:
64                  log.error("instagram failed: %s", e)
65                  db.upsert("publications", {"content_id": s["id"], "platform": "instagram", "status": "failed",
"error": str(e)[:500]}, on="content_id,platform")
66              try:
67                  r = meta.publish_facebook_reel(Path(s["file_path"]), caption)
68                  db.upsert("publications", {"content_id": s["id"], "platform": "facebook", "platform_id":
r["id"], "url": r.get("url"), "privacy": "public",
69                                             "status": "live", "published_at": now_iso()},
on="content_id,platform")
70                  results.append({"platform": "facebook", "content_id": s["id"], **r})
71              except Exception as e:
72                  log.error("facebook failed: %s", e)
73                  db.upsert("publications", {"content_id": s["id"], "platform": "facebook", "status": "failed",
"error": str(e)[:500]}, on="content_id,platform")
74          elif meta.configured():
75              log.info("meta: skipping reels while publish_privacy is private (Reels cannot be scheduled
```

```
private)")
    76
    77      db.update("content", {"id": content_id}, {"status": "published" if results else "failed"})
    78      for s in shorts:
    79          db.update("content", {"id": s["id"]}, {"status": "published" if results else "failed"})
    80      return results
```

== scripts/youtube_auth.py ==

One-time OAuth authorisation by the channel owner

```
   1  """Two-step YouTube OAuth without a local server (works in sandboxes).
   2  Step 1: python scripts/youtube_auth.py url        -> prints the URL to open; log in with the channel's account,
click Allow.
   3  Step 2: the browser lands on http://localhost:8765/?code=... (page will not load; that is expected).
   4          python scripts/youtube_auth.py token "<paste that full URL>"   -> saves YT_REFRESH_TOKEN into .env
   5
   6  Prerequisites: Google Cloud project with YouTube Data API v3 + YouTube Analytics API enabled, OAuth consent
screen
   7  (External, your Gmail as test user), Desktop-app OAuth client; its id/secret in .env as YT_CLIENT_ID /
YT_CLIENT_SECRET.
   8  """
   9  import os
  10  import re
  11  import sys
  12  from pathlib import Path
  13
  14  from dotenv import load_dotenv
  15  from google_auth_oauthlib.flow import Flow
  16
  17  ROOT = Path(__file__).resolve().parent.parent
  18  load_dotenv(ROOT / ".env")
  19  SCOPES = ["https://www.googleapis.com/auth/youtube", "https://www.googleapis.com/auth/youtube.force-ssl",
"https://www.googleapis.com/auth/yt-analytics.readonly"]
  20  REDIRECT = "http://localhost:8765/"
  21
  22  cid, sec = os.getenv("YT_CLIENT_ID"), os.getenv("YT_CLIENT_SECRET")
  23  if not cid or not sec:
  24      sys.exit("Set YT_CLIENT_ID and YT_CLIENT_SECRET in .env first")
  25
  26  flow = Flow.from_client_config({"installed": {"client_id": cid, "client_secret": sec, "auth_uri":
"https://accounts.google.com/o/oauth2/auth",
  27                                                 "token_uri": "https://oauth2.googleapis.com/token",
"redirect_uris": [REDIRECT]}}, SCOPES, redirect_uri=REDIRECT, autogenerate_code_verifier=False)
  28  cmd = sys.argv[1] if len(sys.argv) > 1 else "url"
  29  if cmd == "url":
  30      url, _ = flow.authorization_url(access_type="offline", prompt="consent", include_granted_scopes="true")
  31      print(url)
  32  elif cmd == "token":
  33      pasted = sys.argv[2].strip()
  34      code = re.search(r"[?&]code=([^&]+)", pasted)
  35      if not code:
  36          sys.exit("Could not find code= in the pasted URL")
  37      import os as _os
  38      _os.environ["OAUTHLIB_INSECURE_TRANSPORT"] = "1"  # redirect is http://localhost
  39      flow.fetch_token(code=code.group(1).replace("%2F", "/"))
  40      rt = flow.credentials.refresh_token
  41      if not rt:
  42          sys.exit("No refresh token returned; remove the app's access at myaccount.google.com/permissions and
retry")
  43      env = (ROOT / ".env").read_text()
  44      env = re.sub(r"^YT_REFRESH_TOKEN=.*$", f"YT_REFRESH_TOKEN={rt}", env, flags=re.M) if "YT_REFRESH_TOKEN=" in
env else env + f"\nYT_REFRESH_TOKEN={rt}\n"
  45      (ROOT / ".env").write_text(env)
  46      print("YT_REFRESH_TOKEN saved to .env")
```

== .github/workflows/produce.yml ==

Scheduled run that produces and uploads

```
   1  name: produce
   2  on:
   3    schedule:
   4      - cron: "0 6 * * *"    # 06:00 UTC = 11:00 PKT
   5      - cron: "0 18 * * *"   # 18:00 UTC = 23:00 PKT
   6    workflow_dispatch:
   7      inputs:
   8        privacy:
   9          description: "private | public | unlisted (default: dashboard setting)"
  10          required: false
  11        count:
  12          description: "long videos this run"
  13          required: false
  14          default: "1"
  15  concurrency: produce
```

```yaml
jobs:
  produce:
    runs-on: ubuntu-latest
    timeout-minutes: 90
    env:
      ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
      SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
      SUPABASE_SERVICE_KEY: ${{ secrets.SUPABASE_SERVICE_KEY }}
      PEXELS_API_KEY: ${{ secrets.PEXELS_API_KEY }}
      PIXABAY_API_KEY: ${{ secrets.PIXABAY_API_KEY }}
      YT_CLIENT_ID: ${{ secrets.YT_CLIENT_ID }}
      YT_CLIENT_SECRET: ${{ secrets.YT_CLIENT_SECRET }}
      YT_REFRESH_TOKEN: ${{ secrets.YT_REFRESH_TOKEN }}
      META_ACCESS_TOKEN: ${{ secrets.META_ACCESS_TOKEN }}
      META_PAGE_ID: ${{ secrets.META_PAGE_ID }}
      META_IG_USER_ID: ${{ secrets.META_IG_USER_ID }}
      GH_TOKEN: ${{ github.token }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12", cache: pip }
      - run: sudo apt-get update -qq && sudo apt-get install -y -qq ffmpeg fonts-dejavu-core
      - run: pip install -r requirements.txt
      - name: Produce and publish
        run: |
          ARGS="--publish --count ${{ github.event.inputs.count || '1' }}"
          if [ -n "${{ github.event.inputs.privacy }}" ]; then ARGS="$ARGS --privacy ${{
github.event.inputs.privacy }}"; fi
          python -m pipeline.run_all $ARGS
      - name: Keep the rendered files for 3 days
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: videos-${{ github.run_id }}
          path: |
            out/**/*.mp4
            out/**/*.jpg
            out/**/script.json
            out/**/result.json
          retention-days: 3
          if-no-files-found: ignore
      - name: Persist persona learnings
        if: always()
        run: |
          git config user.name "pipeline-bot" && git config user.email "bot@users.noreply.github.com"
          git add persona || true
          git diff --cached --quiet || (git commit -m "persona: automated update" && git push)
```